

Project Report: Humor Detection

Capstone Project

Introduction

I chose the "UR-Funny" dataset because I wanted to explore how computers can understand human humor and distinguish between funny and serious text.

Humor detection is a fascinating and challenging task in Natural Language Processing (NLP) because humor often depends on subtle wordplay, context, and cultural nuances. This project allowed me to apply various machine learning and deep learning techniques to see which algorithm can capture these text patterns most effectively.

Dataset Description

The dataset used in this project focuses on humor classification. During the data exploration phase, I checked the structure and quality of the data:

- **Shape:** I analyzed the total number of rows and columns to understand the dataset's size.
- **Missing Values:** I checked for any empty or missing data points to ensure the dataset was clean before training.
- **Class Balance:** I investigated the distribution of the target labels to see if the classes were balanced. The results showed a standard distribution across the classes, giving a clear baseline for evaluating our models using standard metrics.

Models Used

To establish a baseline, I trained three different traditional machine learning models using features extracted through the TF-IDF vectorization method:

1. **Logistic Regression:** This model achieved an Accuracy of 63.72% and an F1-Score of 0.333.
2. **Random Forest:** This ensemble model performed the best among traditional algorithms, reaching an Accuracy of 65.73% and an F1-Score of 0.511.

3. **K-Nearest Neighbors (KNN):** This distance-based model achieved an Accuracy of 54.28% and an F1-Score of 0.492.

For each model, I calculated the accuracy, precision, recall, and F1-score, and plotted a confusion matrix to visualize the true positives, true negatives, false positives, and false negatives. Among these three, **Random Forest was the best traditional model** because it handled the high-dimensional text features efficiently.

Neural Network

I built and trained a Deep Learning model using Keras for binary classification. The architecture consists of fully connected Dense layers with ReLU activation for hidden layers, and a Sigmoid function for the output layer. To prevent overfitting, a Dropout layer was implemented. The model was trained for over 10 epochs, and I plotted the training versus validation loss and accuracy curves to monitor performance and ensure learning stability.

Results & Comparison

After compiling the results into a summary table, **Random Forest achieved the highest accuracy (65.73%)**, while the **Neural Network achieved the highest F1-Score (0.6081)**.

The Random Forest model outperformed the Neural Network in raw accuracy for two main reasons:

1. **Data Sparsity:** Random Forest splits effectively on high-dimensional, sparse TF-IDF text matrices, whereas basic dense neural networks struggle with zero-heavy data.
2. **Model Context:** Traditional dense networks lack sequential word context (like embeddings) required to completely outperform tree-based models on raw text frequencies.

What I Learned

This capstone project taught me that more complex models are not always automatically better for every dataset format. I learned that data preparation, text cleaning, and choosing the right feature extraction method (like TF-IDF) play a huge role in the success of an NLP pipeline. Additionally, I learned how to read training curves to balance a model using Dropout layers, and realized that evaluating a model requires looking at both Accuracy and F1-Score together to get the true picture of its performance.

Sultana AL-khlefa